Compte rendu

ProjetSportAP – AP SportSio, Mission C

Application C# WinForms de gestion de sportifs – BTS SIO

1. Présentation

Sport'SIO est une application de bureau en C# (Windows Forms) qui permet de consulter et de gérer les sportifs d'une association, ainsi que les sports qu'ils pratiquent et leur niveau d'expérience. Les données sont stockées dans une base MySQL nommée BdSport et accessibles par ADO.NET. L'accès à l'application est protégé par un login et un mot de passe.

2. Architecture

La solution est découpée en trois projets, selon une approche objet :

Projet	Rôle
APProjetSport	Interface Windows Forms : Program, FormConnexion, FormConnexionCompte, AGestion, FormChangementMdp
BiblioSportif	Classes métier : Sportif, Sport, Niveau, Pratique
GestionSportif	Accès aux données : ConnexionManager, GSportif, GUtilisateur

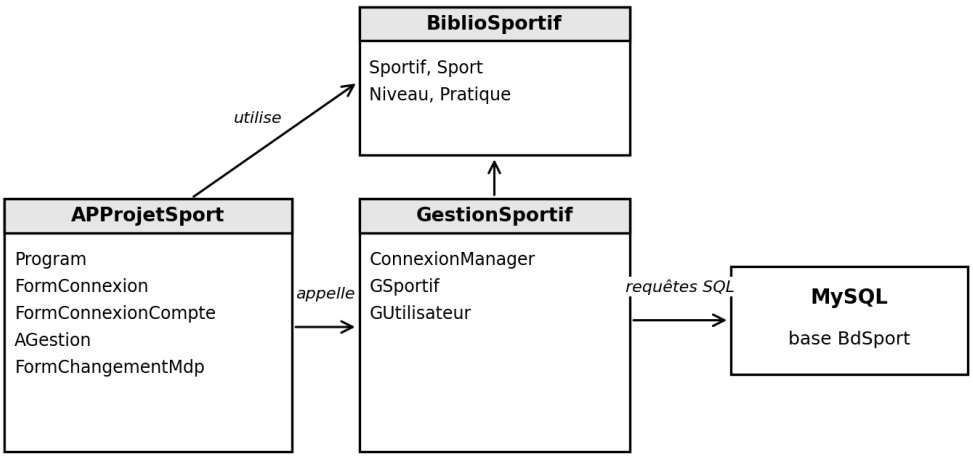


Figure 1 – Architecture de la solution

La chaîne de connexion est lue dans le fichier de configuration (clé cnxBdSport) par la classe ConnexionManager :

```
public static MySqlConnection GetConnection()
{
    string chaineconnexion = ConfigurationManager
        .ConnectionStrings["cnxBdSport"].ConnectionString;
    return new MySqlConnection(chaineconnexion);
}
```

3. Base de données

La base BdSport comporte cinq tables. Un sportif peut pratiquer plusieurs sports, chacun avec un niveau : la table Pratique porte donc cette association, avec pour clé primaire le couple (idSportif, idSport), ce qui interdit les doublons.

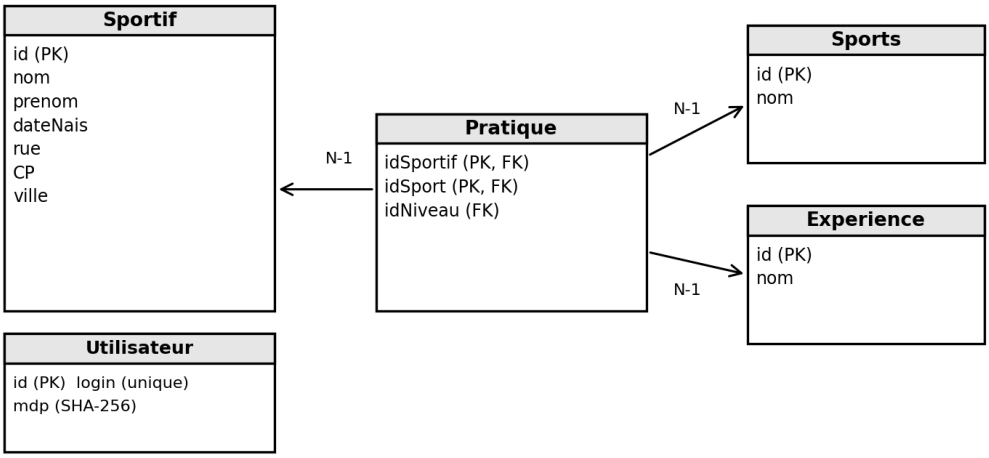


Figure 2 – Tables de la base BdSport

Table	Contenu
Sportif	Identité, date de naissance et adresse du sportif
Sports	Liste des sports proposés (Football, Tennis, Judo, Natation, Basketball, Rugby, Course)
Experience	Niveaux : Débutant, Confirmé, Avancé, Professionnel
Pratique	Sport et niveau d'un sportif
Utilisateur	Comptes de connexion (login unique, mot de passe haché)

4. Fonctionnalités

4.1 Connexion

Au lancement, Program affiche le formulaire de connexion. L'application principale (AGestion) n'est ouverte que si la connexion réussit ; le login est alors transmis au formulaire principal.

```
using (FormConnexion connexion = new FormConnexion())
{
    if (connexion.ShowDialog() == DialogResult.OK)
    {
        Application.Run(new AGestion(connexion.Login));
    }
}
```

Les mots de passe ne sont jamais stockés en clair : ils sont hachés en SHA-256 par MySQL, et les requêtes sont paramétrées.

```
MySqlCommand cmd = new MySqlCommand(
    "SELECT COUNT(*) FROM Utilisateur " +
    "WHERE login = @login AND mdp = SHA2(@mdp, 256)", cnx);
cmd.Parameters.AddWithValue("@login", login);
cmd.Parameters.AddWithValue("@mdp", mdp);
return Convert.ToInt32(cmd.ExecuteScalar()) > 0;
```

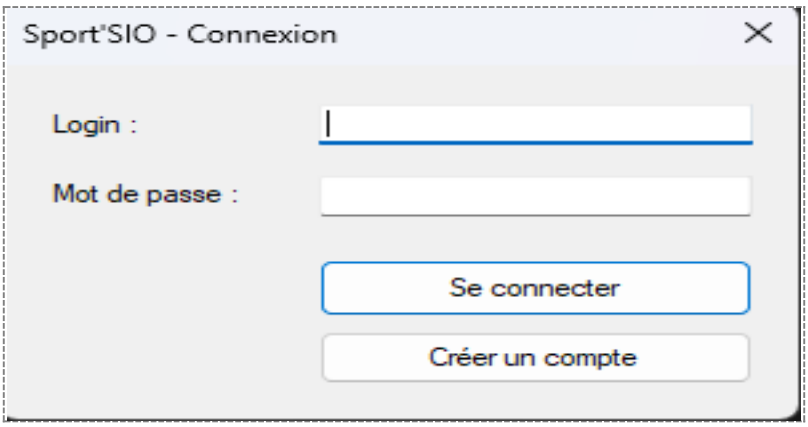


Figure 3 – Formulaire de connexion (FormConnexion)

4.2 Création de compte et changement de mot de passe

Depuis la connexion, le bouton « Créer un compte » ouvre un formulaire dédié. Les contrôles effectués sont les suivants :

- login d'au moins 3 caractères, et non déjà utilisé ;
- mot de passe d'au moins 6 caractères ;
- confirmation identique au mot de passe.

Le changement de mot de passe (bouton dans la fenêtre principale) applique les mêmes règles et exige l'ancien mot de passe : la requête de mise à jour ne modifie la ligne que si celui-ci est correct.

```
"UPDATE Utilisateur SET mdp = SHA2(@nouveau, 256) " +
"WHERE login = @login AND mdp = SHA2(@ancien, 256)"
```

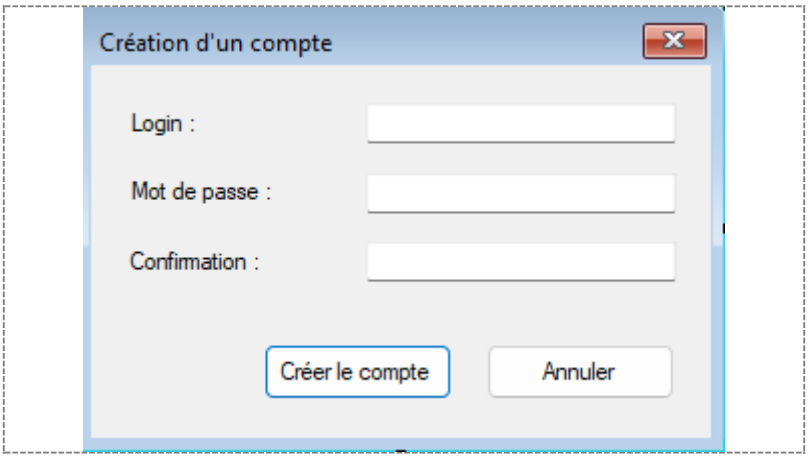


Figure 4 – Création de compte (FormConnexionCompte)

4.3 Onglet Consultation

Cet onglet affiche tous les sportifs dans une liste (nom, prénom, date de naissance, adresse, sports pratiqués). Une recherche est possible selon trois critères : la ville, le niveau d'expérience ou le nom du sport. Un clic sur un sportif affiche le détail de sa fiche et de tous ses sports.

```
switch (CbListeDeroulante.SelectedIndex)
{
    case 0:     // Ville
        correspond = sportif.Ville.ToLower().Contains(motRecherche);
        break;
    case 1:     // Niveau : au moins une pratique avec ce niveau
        ...
    case 2:     // Sport : au moins une pratique de ce sport
        ...
}
```

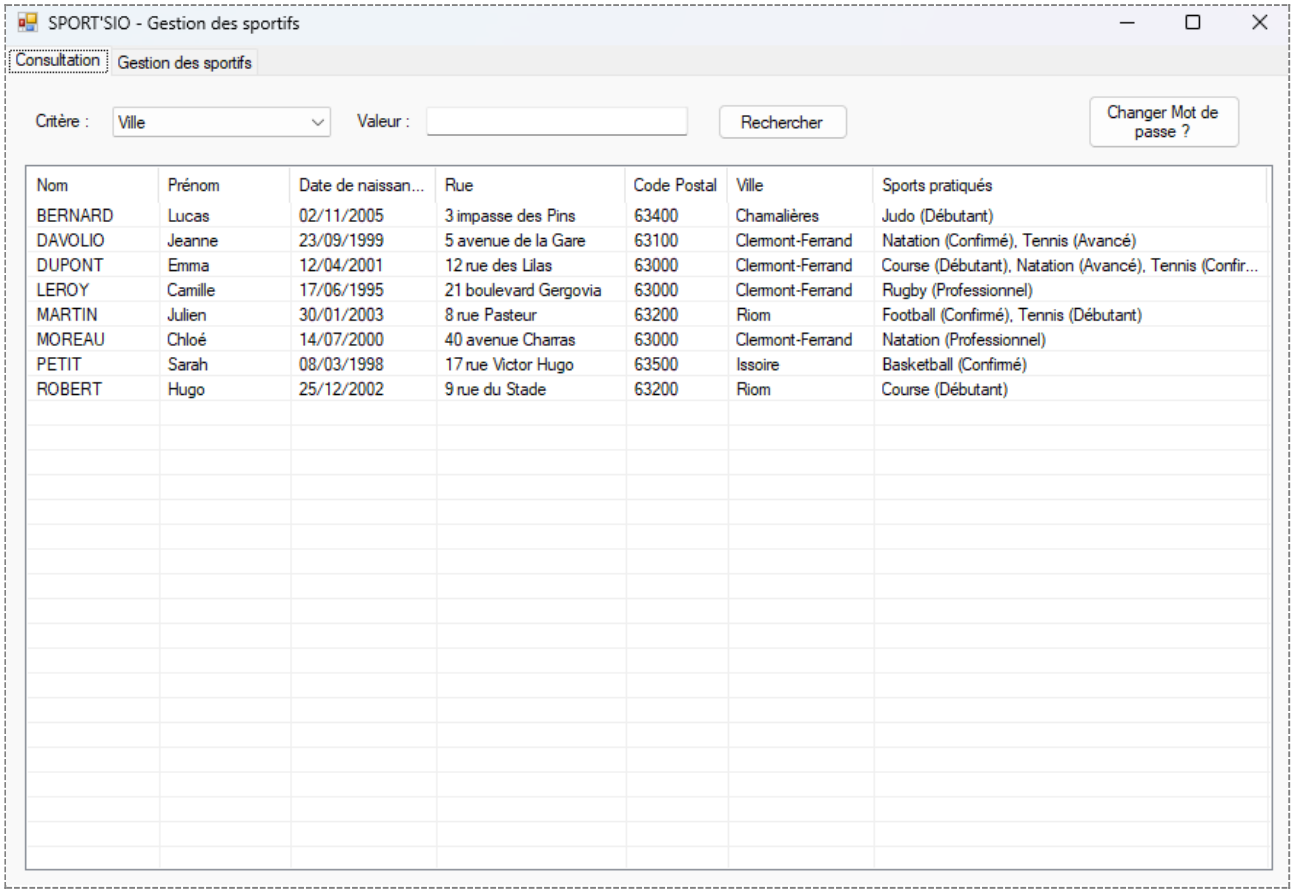


Figure 5 – Onglet Consultation (AGestion)

4.4 Onglet Gestion des sportifs

Cet onglet permet d'ajouter, de modifier et de supprimer un sportif à l'aide d'une fiche. Le bouton « Vider le formulaire » remet la fiche à zéro. La suppression demande une confirmation et entraîne la suppression des sports pratiqués du sportif (suppression en cascade).

Avant toute écriture en base, la saisie est contrôlée :

- nom, prénom, rue et ville obligatoires ;
- date de naissance antérieure à aujourd'hui ;
- code postal composé d'exactement 5 chiffres.

```
if (!Regex.IsMatch(txtCP.Text.Trim(), @"^\d{5}$"))
{
    erreurs += "- Le code postal doit contenir exactement 5 chiffres.\n";
}
```

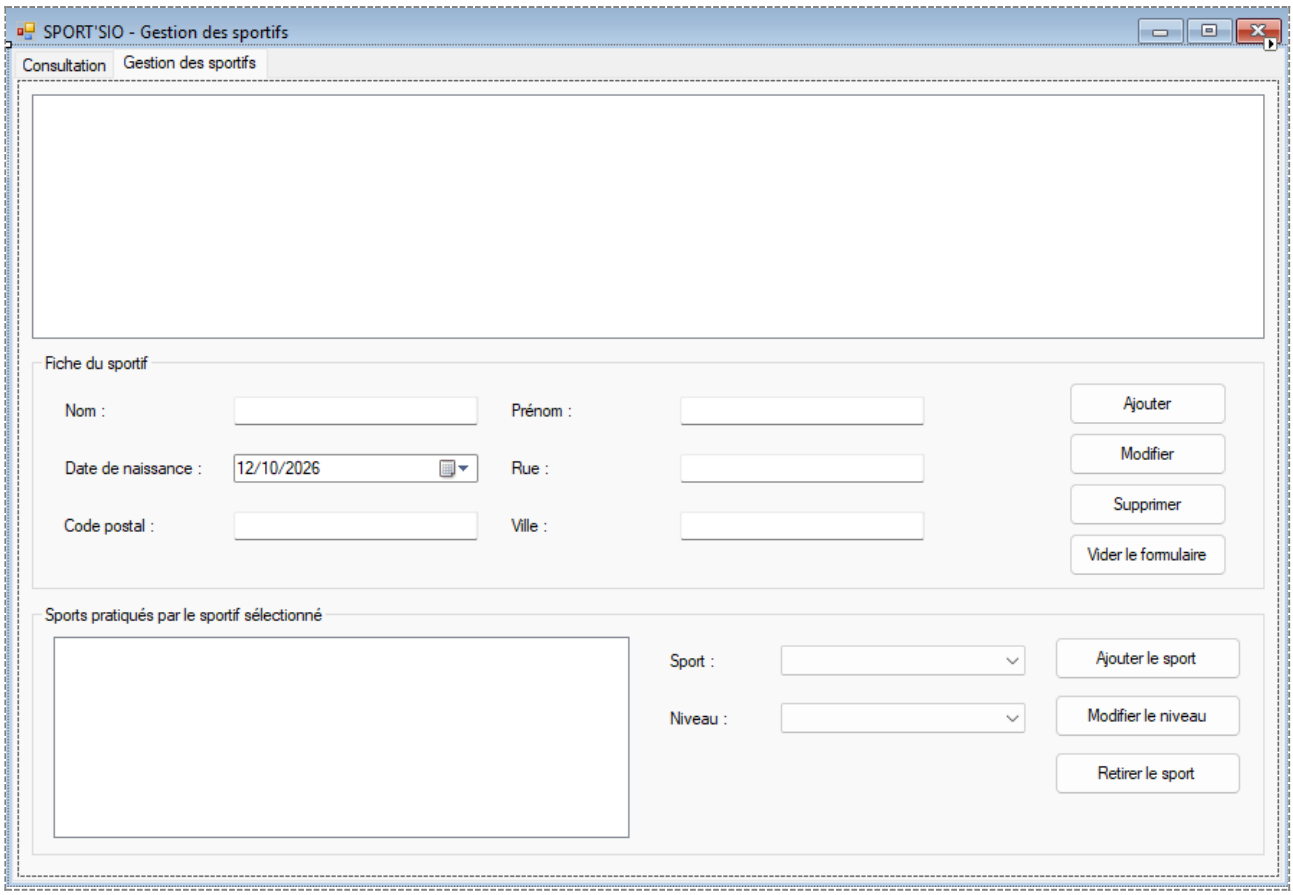


Figure 6 – Onglet Gestion des sportifs (AGestion)

4.5 Sports pratiqués par un sportif

Lorsqu'un sportif est sélectionné, le bloc « Sports pratiqués » devient actif. Il permet d'ajouter un sport avec son niveau, de modifier le niveau d'un sport, ou de retirer un sport (seule la ligne de Pratique est supprimée, ni le sportif ni le sport). Une règle de gestion empêche d'ajouter deux fois le même sport au même sportif :

```
public bool PratiqueSport(int idSport)
{
    for (int i = 0; i < pratiques.Count; i++)
    {
        if (pratiques[i].Sport.Id == idSport)
        {
            return true;
        }
    }
    return false;
}
```

Les sports et niveaux proposés dans les listes déroulantes sont lus dans la base au chargement du formulaire.

5. Gestion des erreurs

Tous les accès à la base sont encadrés par un bloc try/catch : en cas d'erreur, un message est affiché à l'utilisateur sans interrompre l'application. Les actions sensibles (suppression d'un sportif, retrait d'un sport) demandent une confirmation.

6. Conclusion

L'application répond au besoin de gestion des sportifs : accès sécurisé, consultation avec recherche, gestion complète des sportifs et de leurs sports pratiqués, le tout organisé en trois couches (interface, classes métier, accès aux données).